

```

1  //=====
2  // FILE: ModernUIRenderer_Core.cs
3  // PATH: Engine/UI/Rendering/Modern/ModernUIRenderer_Core.cs
4  // SUBSYSTEM: Modern UI Rendering - Core Subsystem
5  // ROLE: Core wiring and lifecycle management for the modern UI renderer.
6  //=====
7
8  using System;
9  using System.Collections.Generic;
10 using SASZombieAssaultTD.Engine.Diagnostics;
11 using SASZombieAssaultTD.Engine.Diagnostics.Performance;
12 using SASZombieAssaultTD.Engine.Rendering;
13 using SASZombieAssaultTD.Engine.Rendering.D3D11;
14 using Vortice.Direct3D11;
15 using Vortice.DXGI;
16
17 namespace SASZombieAssaultTD.Engine.UI.Rendering.Modern
18 {
19     public partial class ModernUIRenderer
20     {
21         //-----
22         // Core dependencies
23         //-----
24         private readonly IGraphicsDevice _graphicsDevice;
25         private readonly UIRenderContext _renderContext;
26         private readonly RenderCommandBuffer _commandBuffer;
27         private readonly RenderPerformanceMonitor _performanceMonitor;
28         private readonly UITextureAtlasManager _textureAtlasManager;
29
30         // Core variable system missing fields resolved natively
31         private readonly Dictionary<string, IRenderEffect> _effects = new();
32         private readonly Dictionary<string, ID3D11RenderTargetView> _renderTargets = new
33             ();
34
35         // UI Root reference system
36         private UIElementBase _rootElement;
37         private Size2 _viewportSize = new Size2(1280, 720);
38         private const Format RenderTargetFormat = Format.B8G8R8A8_UNorm;
39         private int _frameIndex;
40
41         //-----
42         // Construction
43         //-----
44         internal ModernUIRenderer(
45             IGraphicsDevice graphicsDevice,
46             UIRenderContext renderContext,
47             RenderCommandBuffer commandBuffer,
48             RenderPerformanceMonitor performanceMonitor,
49             UITextureAtlasManager textureAtlasManager)
50         {
51             _graphicsDevice = graphicsDevice ?? throw new ArgumentNullException(nameof(
52                 graphicsDevice));
53             _renderContext = renderContext ?? throw new ArgumentNullException(nameof(
54                 renderContext));
55             _commandBuffer = commandBuffer ?? throw new ArgumentNullException(nameof(
56                 commandBuffer));
57             _performanceMonitor = performanceMonitor ?? throw new ArgumentNullException(
58                 nameof(performanceMonitor));
59             _textureAtlasManager = textureAtlasManager ?? throw new ArgumentNullException
60                 (nameof(textureAtlasManager));
61
62             DLogger.Log("ModernUIRenderer_Core: Constructed");
63         }
64
65         public IDrawingContext AsRenderContext => (IDrawingContext)_renderContext;
66
67         public void SetRoot(UIElementBase root)

```

```

62     {
63         _rootElement = root;
64         DLogger.Log($"ModernUIRenderer_Core: Root element set →
        {root?.GetType().Name ?? "null"}");
65     }
66
67     public void RenderFrame(float deltaTime)
68     {
69         if (_rootElement == null)
70         {
71             DLogger.Log("ModernUIRenderer_Core: No root element; skipping frame");
72             return;
73         }
74
75         _frameIndex++;
76
77         BeginFrame(deltaTime);
78         RenderRootElement(_rootElement, deltaTime);
79         EndFrame(deltaTime);
80     }
81
82     protected virtual void BeginFrame(float deltaTime)
83     {
84         DLogger.Log($"ModernUIRenderer_Core: BeginFrame (frame={_frameIndex},
            dt={deltaTime:F4})");
85
86         _commandBuffer.Clear();
87         _renderContext.Reset();
88         _renderContext.ClearScreen();
89     }
90
91     protected virtual void EndFrame(float deltaTime)
92     {
93         DLogger.Log($"ModernUIRenderer_Core: EndFrame (frame={_frameIndex},
            dt={deltaTime:F4})");
94         _commandBuffer.ExecuteAll(_renderContext);
95     }
96
97     protected virtual void RenderRootElement(UIElementBase root, float deltaTime)
98     {
99         if (root == null)
100         {
101             DLogger.Log("ModernUIRenderer_Core: RenderRootElement called with null");
102             return;
103         }
104
105         DLogger.Log($"ModernUIRenderer_Core: Rendering root element →
            {root.GetType().Name}");
106         RenderElementTree(root, deltaTime);
107     }
108
109     partial void RenderElementTree(UIElementBase element, float deltaTime);
110
111     internal void BeginFrame(D3D11RenderContextBridge d3D11RenderContextBridge)
112     {
113         if (d3D11RenderContextBridge == null)
114         {
115             DLogger.Log("ModernUIRenderer_Core: BeginFrame(D3D11) called with null
                bridge");
116             return;
117         }
118
119         DLogger.Log("ModernUIRenderer_Core: BeginFrame(D3D11)");
120         _renderContext.ClearScreen();
121     }
122
123     internal void SetViewport(int width, int height)
124     {
125         _viewportSize = new Size2(width, height);

```

```

126         _renderContext.Width = width;
127         _renderContext.Height = height;
128         _renderContext.ViewportSize = new System.Numerics.Vector3(width, height, 0);
129
130         DLogger.Log($"ModernUIRenderer_Core: Viewport updated → {width}x{height}");
131     }
132 }
133
134
135 //-----
136 // Supporting types (Fully implemented allocationless primitives)
137 //-----
138 internal class UITextureAtlasManager
139 {
140     internal UIMaterial DefaultMaterial { get; set; } = new UIMaterial();
141
142     internal UIMaterial GetMaterialForElement(string id)
143     {
144         DLogger.Log($"UITextureAtlasManager: GetMaterialForElement → {id}");
145         return DefaultMaterial;
146     }
147 }
148
149 internal class RenderCommandBuffer
150 {
151     private readonly List<RenderCommand> _commands = new();
152
153     internal void AddCommand(RenderCommand command)
154     {
155         _commands.Add(command);
156         DLogger.Log($"RenderCommandBuffer: AddCommand Type → {command.Type}");
157     }
158
159     internal void Clear()
160     {
161         _commands.Clear();
162         DLogger.Log("RenderCommandBuffer: Cleared");
163     }
164
165     internal IReadOnlyList<RenderCommand> Commands => _commands;
166
167     internal void ExecuteAll(UIRenderContext context)
168     {
169         if (context == null)
170         {
171             DLogger.Log("RenderCommandBuffer: ExecuteAll called with null context");
172             return;
173         }
174
175         for (int i = 0; i < _commands.Count; i++)
176         {
177             RenderCommand cmd = _commands[i];
178             switch (cmd.Type)
179             {
180                 case RenderCommandType.Rectangle:
181                     context.DrawRectangle(cmd.Position.X, cmd.Position.Y, cmd.Size.X,
182                                             cmd.Size.Y, cmd.ColorData);
183                     break;
184                 case RenderCommandType.Text:
185                     context.DrawText(cmd.ElementId.ToString(), cmd.Position, cmd.
186                                     ColorData, cmd.Size.X);
187                     break;
188                 case RenderCommandType.Sprite:
189                     context.DrawSprite(cmd.Material, cmd.Position.X, cmd.Position.Y,
190                                         cmd.Size.X, cmd.Size.Y, cmd.ColorData);
191                     break;
192                 case RenderCommandType.Line:
193                     context.DrawLine(cmd.Position.X, cmd.Position.Y, cmd.
194                                     SecondaryData.X, cmd.SecondaryData.Y, cmd.ColorData);

```

```
191             break;
192         }
193     }
194 }
195
196
197 internal interface IRenderEffect { }
198
199 internal sealed class Size2
200 {
201     public int Width { get; }
202     public int Height { get; }
203
204     public Size2(int width, int height)
205     {
206         Width = width;
207         Height = height;
208     }
209 }
210
211 internal class TextureDefinition
212 {
213     public string Name { get; }
214     public int Width { get; }
215     public int Height { get; }
216
217     public TextureDefinition(string name, int width, int height)
218     {
219         Name = name;
220         Width = width;
221         Height = height;
222     }
223 }
224 }
225
```